EOSC 213: Computational Methods in Geological Engineering

Lecture 13: Boundary Value Problems (BVPs) + Finite Differences for BVPs

Shadab Ahamed

Department of Earth, Ocean and Atmospheric Sciences,
The University of British Columbia

Tue, Mar 3, 2026

Correction: Eigenvalues of triangular matrices

Both **upper** and **lower** triangular matrices have eigenvalues equal to their diagonal entries.

Correction: Eigenvalues of triangular matrices

Both **upper** and **lower** triangular matrices have eigenvalues equal to their diagonal entries.

Upper triangular matrix

$$U = \begin{bmatrix} \mu_1 & * & * \\ 0 & \mu_2 & * \\ 0 & 0 & \mu_3 \end{bmatrix} \implies \det(U - \lambda I) = (\lambda - \mu_1)(\lambda - \mu_2)(\lambda - \mu_3) = 0.$$

Correction: Eigenvalues of triangular matrices

Both **upper** and **lower** triangular matrices have eigenvalues equal to their diagonal entries.

Upper triangular matrix

$$U = \begin{bmatrix} \mu_1 & * & * \\ 0 & \mu_2 & * \\ 0 & 0 & \mu_3 \end{bmatrix} \implies \det(U - \lambda I) = (\lambda - \mu_1)(\lambda - \mu_2)(\lambda - \mu_3) = 0.$$

Lower triangular matrix

$$L = \begin{bmatrix} \mu_1 & 0 & 0 \\ * & \mu_2 & 0 \\ * & * & \mu_3 \end{bmatrix}$$

Correction: Eigenvalues of triangular matrices

Both **upper** and **lower** triangular matrices have eigenvalues equal to their diagonal entries.

Upper triangular matrix

$$U = \begin{bmatrix} \mu_1 & * & * \\ 0 & \mu_2 & * \\ 0 & 0 & \mu_3 \end{bmatrix} \implies \det(U - \lambda I) = (\lambda - \mu_1)(\lambda - \mu_2)(\lambda - \mu_3) = 0.$$

Lower triangular matrix

$$L = \begin{bmatrix} \mu_1 & 0 & 0 \\ * & \mu_2 & 0 \\ * & * & \mu_3 \end{bmatrix}$$

Observe, the transpose (rows $\leftrightarrow$ columns) of L is upper-triangular,

$$L^\top = \begin{bmatrix} \mu_1 & * & * \\ 0 & \mu_2 & * \\ 0 & 0 & \mu_3 \end{bmatrix}$$

Correction: Eigenvalues of triangular matrices

Lower triangular matrix

$$L = \begin{bmatrix} \mu_1 & 0 & 0 \\ * & \mu_2 & 0 \\ * & * & \mu_3 \end{bmatrix}, \qquad L^\top = \begin{bmatrix} \mu_1 & * & * \\ 0 & \mu_2 & * \\ 0 & 0 & \mu_3 \end{bmatrix} \quad \text{(upper triangular).}$$

Correction: Eigenvalues of triangular matrices

Lower triangular matrix

$$L = \begin{bmatrix} \mu_1 & 0 & 0 \\ * & \mu_2 & 0 \\ * & * & \mu_3 \end{bmatrix}, \qquad L^\top = \begin{bmatrix} \mu_1 & * & * \\ 0 & \mu_2 & * \\ 0 & 0 & \mu_3 \end{bmatrix} \quad \text{(upper triangular).}$$

Key facts

- $\det(A) = \det(A^\top)$

Correction: Eigenvalues of triangular matrices

Lower triangular matrix

$$L = \begin{bmatrix} \mu_1 & 0 & 0 \\ * & \mu_2 & 0 \\ * & * & \mu_3 \end{bmatrix}, \qquad L^\top = \begin{bmatrix} \mu_1 & * & * \\ 0 & \mu_2 & * \\ 0 & 0 & \mu_3 \end{bmatrix} \quad \text{(upper triangular).}$$

Key facts

- $\det(A) = \det(A^\top)$
- Eigenvalues satisfy $\det(A - \lambda I) = 0$

Correction: Eigenvalues of triangular matrices

Lower triangular matrix

$$L = \begin{bmatrix} \mu_1 & 0 & 0 \\ * & \mu_2 & 0 \\ * & * & \mu_3 \end{bmatrix}, \qquad L^\top = \begin{bmatrix} \mu_1 & * & * \\ 0 & \mu_2 & * \\ 0 & 0 & \mu_3 \end{bmatrix} \quad \text{(upper triangular).}$$

Key facts

- $\det(A) = \det(A^\top)$
- Eigenvalues satisfy $\det(A - \lambda I) = 0$

Therefore,

$$\det(L - \lambda I) = \det(L^\top - \lambda I),$$

so the eigenvalues of L are also its diagonal entries.

Correction: Eigenvalues of triangular matrices

Lower triangular matrix

$$L = \begin{bmatrix} \mu_1 & 0 & 0 \\ * & \mu_2 & 0 \\ * & * & \mu_3 \end{bmatrix}, \qquad L^\top = \begin{bmatrix} \mu_1 & * & * \\ 0 & \mu_2 & * \\ 0 & 0 & \mu_3 \end{bmatrix} \quad \text{(upper triangular).}$$

Key facts

- $\det(A) = \det(A^\top)$
- Eigenvalues satisfy $\det(A - \lambda I) = 0$

Therefore,

$$\det(L - \lambda I) = \det(L^\top - \lambda I),$$

so the eigenvalues of L are also its diagonal entries.

Triangular matrices reveal their eigenvalues directly on the diagonal.

Logistics: Midterm I solutions

Midterm I solutions are now posted on Canvas.

Please:

- Carefully review the full solutions.
- Compare with your own approach — not just final answers.
- Identify where mistakes occurred:
  - conceptual understanding,
  - algebra/calculation errors,
  - numerical method setup.

Goal: use the midterm as feedback for improvement before Midterm II and the Final Exam.

Logistics: TA Office Hours + Course Feedback Survey

TA Office Hours Update

To better match student demand:

- TA office hours will now be held **once every two weeks** (since nobody attends, LOL).
- TA will announce exact schedule on Canvas.

Logistics: TA Office Hours + Course Feedback Survey

TA Office Hours Update

To better match student demand:

- TA office hours will now be held **once every two weeks** (since nobody attends, LOL).
- TA will announce exact schedule on Canvas.

Mid-Course Student Feedback

- Please fill out the short survey posted on Canvas.
- Your responses will help us improve the course content as well as how it is delivered and your experience taking this course.

Recap: What is an ODE?

An **ordinary differential equation (ODE)** relates a function and its derivatives,

$$y'(t) = f(t, y).$$

Recap: What is an ODE?

An **ordinary differential equation (ODE)** relates a function and its derivatives,

$$y'(t) = f(t, y).$$

In EOSC 213, we studied ODEs as models of:

- physical systems evolving in time
- so far, we have only covered first-order ODEs in detail

Recap: What is an ODE?

An **ordinary differential equation (ODE)** relates a function and its derivatives,

$$y'(t) = f(t, y).$$

In EOSC 213, we studied ODEs as models of:

- physical systems evolving in time
- so far, we have only covered first-order ODEs in detail

Goal: find $y(t)$ either **analytically** (if possible, usually it is not) or **numerically** (what we learn in this course).

Recap: Initial Value Problems (IVPs)

Most problems so far were,

$$y'(t) = f(t, y), \qquad y(t_0) = y_0.$$

Recap: Initial Value Problems (IVPs)

Most problems so far were,

$$y'(t) = f(t, y), \qquad y(t_0) = y_0.$$

Key idea:

- The solution is fully determined by the initial state.
- We compute the solution by moving forward in time.

Known start $\Rightarrow$ evolve forward.

Recap: Numerical time-stepping methods

We replaced first-order derivatives with their approximations.

Recap: Numerical time-stepping methods

We replaced first-order derivatives with their approximations.

Forward Euler

$$y_{n+1} = y_n + hf(t_n, y_n)$$

Recap: Numerical time-stepping methods

We replaced first-order derivatives with their approximations.

Forward Euler

$$y_{n+1} = y_n + hf(t_n, y_n)$$

Backward Euler

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Recap: Numerical time-stepping methods

We replaced first-order derivatives with their approximations.

Forward Euler

$$y_{n+1} = y_n + hf(t_n, y_n)$$

Backward Euler

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Explicit Midpoint

$$y_{n+1} = y_n + hf\left(t_n + \frac{h}{2},\, y_n + \frac{h}{2} f(t_n, y_n)\right)$$

Recap: Numerical time-stepping methods

We replaced first-order derivatives with their approximations.

Forward Euler

$$y_{n+1} = y_n + hf(t_n, y_n)$$

Backward Euler

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Explicit Midpoint

$$y_{n+1} = y_n + hf\left(t_n + \frac{h}{2},\, y_n + \frac{h}{2} f(t_n, y_n)\right)$$

All are based on approximating derivatives locally.

Recap: Accuracy and stability of numerical methods

We learned that numerical methods must satisfy:

Accuracy

- local truncation error
- order of convergence

Stability

- small errors should not grow uncontrollably
- depends on step size h

Recap: Systems of ODEs

Many physical problems involve multiple variables,

$$\mathbf{y}'(t) = \mathbf{f}(t, \mathbf{y}).$$

We rewrote systems using matrices,

$$\mathbf{y}' = A\mathbf{y}.$$

Key concepts:

- coupled dynamics
- eigenvalues and eigenvectors
- phase-space intuition

Recap: Implementation viewpoint

Numerical solution workflow:

1. Choose time grid
2. Approximate derivatives
3. Update solution step-by-step
4. Analyze stability and behavior

Recap: Implementation viewpoint

Numerical solution workflow:

1. Choose time grid
2. Approximate derivatives
3. Update solution step-by-step
4. Analyze stability and behavior

Important observation:

All numerical methods that we discussed so far compute the solution *sequentially*.

$$y_0 \to y_1 \to y_2 \to \cdots$$

Recap: ODEs as *time evolution* (IVPs)

What we have done so far,

$$y'(t) = f(t, y), \qquad y(t_0) = y_0.$$

Recap: ODEs as *time evolution* (IVPs)

What we have done so far,

$$y'(t) = f(t, y), \qquad y(t_0) = y_0.$$

IVP idea: the condition at t_0 is enough to "march forward".

Recap: ODEs as *time evolution* (IVPs)

What we have done so far,

$$y'(t) = f(t, y), \qquad y(t_0) = y_0.$$

IVP idea: the condition at t_0 is enough to "march forward".

- Choose step size h
- Use Forward Euler / Midpoint / Backward Euler
- Update $y_0 \to y_1 \to y_2 \to \cdots$

Recap: ODEs as *time evolution* (IVPs)

What we have done so far,

$$y'(t) = f(t, y), \qquad y(t_0) = y_0.$$

IVP idea: the condition at t_0 is enough to "march forward".

- Choose step size h
- Use Forward Euler / Midpoint / Backward Euler
- Update $y_0 \to y_1 \to y_2 \to \cdots$

Key feature: the solution is built *sequentially*.

Today: not all differential equations are time-evolution problems

Many problems are posed on a **spatial domain** $x \in [0, L]$.

Today: not all differential equations are time-evolution problems

Many problems are posed on a **spatial domain** $x \in [0, L]$.

You are told information at multiple locations:

$$y(0) = a, \qquad y(L) = b.$$

These are called **Boundary Value Problems (BVPs)**.

Today: not all differential equations are time-evolution problems

Many problems are posed on a **spatial domain** $x \in [0, L]$.

You are told information at multiple locations:

$$y(0) = a, \qquad y(L) = b.$$

These are called **Boundary Value Problems (BVPs)**.

Question: How do we "march forward" if we are constrained at *both ends*?

Today: not all differential equations are time-evolution problems

Many problems are posed on a **spatial domain** $x \in [0, L]$.

You are told information at multiple locations:

$$y(0) = a, \qquad y(L) = b.$$

These are called **Boundary Value Problems (BVPs)**.

Question: How do we "march forward" if we are constrained at *both ends*?

Answer: We usually cannot. We must solve **globally**.

IVP vs BVP (big picture)

Initial Value Problem (IVP)	**Boundary Value Problem (BVP)**
Given at one point (usually "start time")	Given at multiple points (often boundaries)
Sequential updates (marching)	Global coupling (solve all at once)
Natural for dynamics in time	Natural for equilibrium / steady-state / spatial profiles

IVP vs BVP (big picture)

Initial Value Problem (IVP)	**Boundary Value Problem (BVP)**
Given at one point (usually "start time")	Given at multiple points (often boundaries)
Sequential updates (marching)	Global coupling (solve all at once)
Natural for dynamics in time	Natural for equilibrium / steady-state / spatial profiles

Important: We will study BVPs mostly for **second-order ODEs** since they naturally arise in physics/geophysics. But, BVPs are *not* only for second-order ODEs. They can appear for first-order ODEs too (less common in physics, but valid).

Definition: Boundary Value Problem (BVP)

A **boundary value problem** is an ODE (or system) with constraints specified at **two or more points** in the domain.

Definition: Boundary Value Problem (BVP)

A **boundary value problem** is an ODE (or system) with constraints specified at **two or more points** in the domain.

Typical form (scalar, 2nd order):

$$y''(x) = g(x, y, y'), \qquad y(0) = a,\ y(L) = b.$$

Definition: Boundary Value Problem (BVP)

A **boundary value problem** is an ODE (or system) with constraints specified at **two or more points** in the domain.

Typical form (scalar, 2nd order):

$$y''(x) = g(x, y, y'), \qquad y(0) = a,\ y(L) = b.$$

Common boundary types:

- **Dirichlet:** $y(0) = a$, $y(L) = b$ (value prescribed at the boundary)
- **Neumann:** $y'(0) = \alpha$, $y'(L) = \beta$ (derivative/slope prescribed at the boundary)
- **Mixed/Robin:** $\gamma y + \delta y' = r$ at the boundary

Physical motivations (why BVPs show up)

BVPs arise naturally when the solution must satisfy **constraints at boundaries**.

Physical motivations (why BVPs show up)

BVPs arise naturally when the solution must satisfy **constraints at boundaries**.

- **Steady heat conduction (1D rod):** temperature fixed at both ends
- **Groundwater head / hydraulic potential:** specified at boundaries
- **Elastic beam/deflection:** displacement/slope constraints at supports
- **Diffusion at steady state:** boundary concentrations/fluxes given

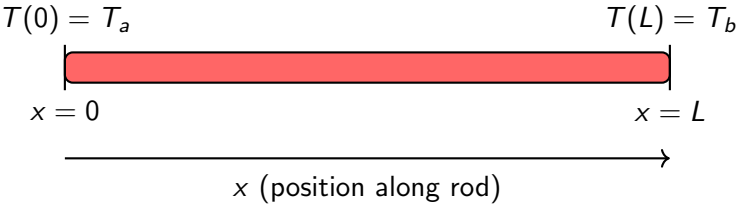
Physical motivations (why BVPs show up)

BVPs arise naturally when the solution must satisfy **constraints at boundaries**.

- **Steady heat conduction (1D rod):** temperature fixed at both ends
- **Groundwater head / hydraulic potential:** specified at boundaries
- **Elastic beam/deflection:** displacement/slope constraints at supports
- **Diffusion at steady state:** boundary concentrations/fluxes given

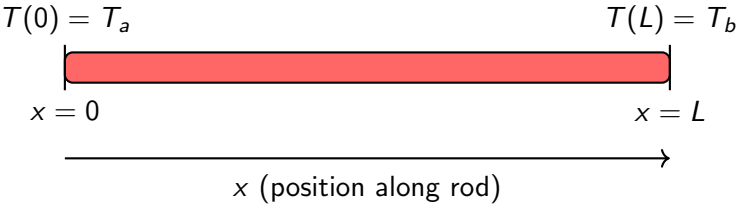
Key point: these are often *equilibrium / steady* problems.

Example 1: Steady heat in 1D (with Dirichlet BC)



Let $T(x)$ be temperature along a rod, $x \in [0, L]$.

Example 1: Steady heat in 1D (with Dirichlet BC)

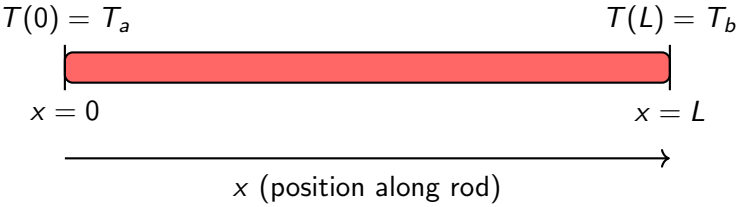


Let $T(x)$ be temperature along a rod, $x \in [0, L]$. Steady state (no time dependence) often leads to,

$$-k\, T''(x) = q(x),$$

where k is conductivity and $q(x)$ is internal heat source.

Example 1: Steady heat in 1D (with Dirichlet BC)



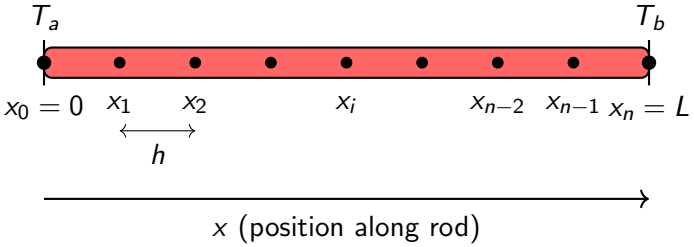
Let $T(x)$ be temperature along a rod, $x \in [0, L]$. Steady state (no time dependence) often leads to,

$$-k\, T''(x) = q(x),$$

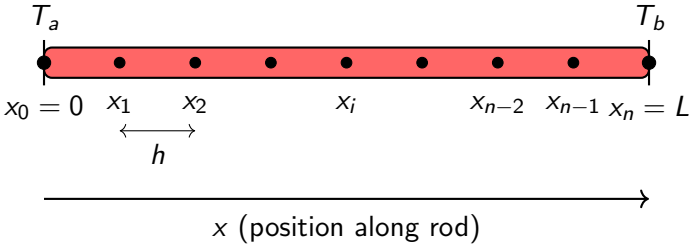
where k is conductivity and $q(x)$ is internal heat source.

Boundary conditions (temperature fixed): $T(0) = T_a, \quad T(L) = T_b$.
This is a **BVP** because constraints are at both ends.

Discretize the domain



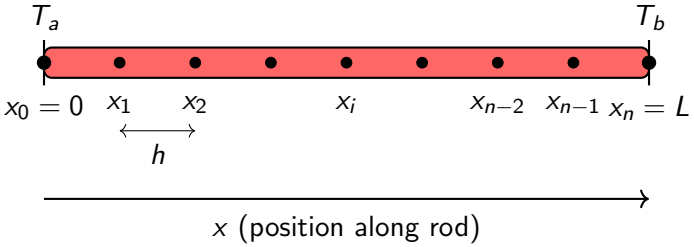
Discretize the domain



Take grid points:

$$x_i = ih, \qquad i = 0, 1, \ldots, n, \qquad h = \frac{L}{n}$$

Discretize the domain



Take grid points:

$$x_i = ih, \qquad i = 0, 1, \ldots, n, \qquad h = \frac{L}{n}$$

Boundary values are known: $T_0 = T_a, \quad T_n = T_b$.
Unknowns are the **interior points:** $T_1, \ldots, T_{n-1}$.

Finite differences: first vs second derivatives

First derivative (used earlier for IVPs)

Finite differences: first vs second derivatives

First derivative (used earlier for IVPs)

Forward difference:

$$T'(x_i) \approx \frac{T_{i+1} - T_i}{h}$$

Finite differences: first vs second derivatives

First derivative (used earlier for IVPs)

Forward difference:

$$T'(x_i) \approx \frac{T_{i+1} - T_i}{h}$$

Backward difference:

$$T'(x_i) \approx \frac{T_i - T_{i-1}}{h}$$

Finite differences: first vs second derivatives

First derivative (used earlier for IVPs)

Forward difference:

$$T'(x_i) \approx \frac{T_{i+1} - T_i}{h}$$

Backward difference:

$$T'(x_i) \approx \frac{T_i - T_{i-1}}{h}$$

Central difference:

$$T'(x_i) \approx \frac{T_{i+1} - T_{i-1}}{2h}$$

Finite differences: first vs second derivatives

First derivative (used earlier for IVPs)

Forward difference:

$$T'(x_i) \approx \frac{T_{i+1} - T_i}{h}$$

Backward difference:

$$T'(x_i) \approx \frac{T_i - T_{i-1}}{h}$$

Central difference:

$$T'(x_i) \approx \frac{T_{i+1} - T_{i-1}}{2h}$$

Second derivative (for BVPs)

Finite differences: first vs second derivatives

First derivative (used earlier for IVPs)

Forward difference:

$$T'(x_i) \approx \frac{T_{i+1} - T_i}{h}$$

Backward difference:

$$T'(x_i) \approx \frac{T_i - T_{i-1}}{h}$$

Central difference:

$$T'(x_i) \approx \frac{T_{i+1} - T_{i-1}}{2h}$$

Second derivative (for BVPs)

For interior points $i = 1, \ldots, n-1$:

$$T''(x_i) \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}.$$

Finite differences: first vs second derivatives

First derivative (used earlier for IVPs)

Forward difference:

$$T'(x_i) \approx \frac{T_{i+1} - T_i}{h}$$

Backward difference:

$$T'(x_i) \approx \frac{T_i - T_{i-1}}{h}$$

Central difference:

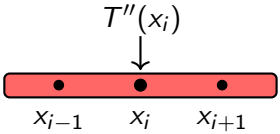
$$T'(x_i) \approx \frac{T_{i+1} - T_{i-1}}{2h}$$

Second derivative (for BVPs)

For interior points $i = 1, \ldots, n-1$:

$$T''(x_i) \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}.$$

Note: *Second derivative uses 3 points (3-point stencil).*



Finite differences for the second derivative (with Dirichlet BC)

For interior points $i = 1, \ldots, n-1$,

$$T''(x_i) \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}.$$

Finite differences for the second derivative (with Dirichlet BC)

For interior points $i = 1, \ldots, n-1$,

$$T''(x_i) \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}.$$

Plug into $-kT''(x) = q(x)$,

$$-k\left(\frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}\right) = q(x_i).$$

Finite differences for the second derivative (with Dirichlet BC)

For interior points $i = 1, \ldots, n-1$,

$$T''(x_i) \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}.$$

Plug into $-kT''(x) = q(x)$,

$$-k\left(\frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}\right) = q(x_i).$$

Rearrange,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Key shift: the unknowns are *coupled*

Notice what happened:

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Key shift: the unknowns are *coupled*

Notice what happened:

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Each equation involves **three unknowns** (T_{i-1}, T_i, T_{i+1}).

Key shift: the unknowns are *coupled*

Notice what happened:

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Each equation involves **three unknowns** (T_{i-1}, T_i, T_{i+1}).

So we do NOT compute T_1 then T_2 then T_3 by marching (like we used to do for IVPs).

Key shift: the unknowns are *coupled*

Notice what happened:

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Each equation involves **three unknowns** (T_{i-1}, T_i, T_{i+1}).

So we do NOT compute T_1 then T_2 then T_3 by marching (like we used to do for IVPs).

Instead, we will solve a **linear system** for all interior values at once.

From finite differences to a linear system (start with equations)

For each interior grid point $i = 1, \ldots, n-1$, we have,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

From finite differences to a linear system (start with equations)

For each interior grid point $i = 1, \ldots, n-1$, we have,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Let's write the **first few** equations explicitly,

From finite differences to a linear system (start with equations)

For each interior grid point $i = 1, \ldots, n-1$, we have,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Let's write the **first few** equations explicitly,

$$\underbrace{-T_0 + 2T_1 - T_2}_{i=1} = \frac{h^2}{k} q(x_1)$$

From finite differences to a linear system (start with equations)

For each interior grid point $i = 1, \ldots, n-1$, we have,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Let's write the **first few** equations explicitly,

$$\underbrace{-T_0 + 2T_1 - T_2}_{i=1} = \frac{h^2}{k} q(x_1)$$

$$\underbrace{-T_1 + 2T_2 - T_3}_{i=2} = \frac{h^2}{k} q(x_2)$$

From finite differences to a linear system (start with equations)

For each interior grid point $i = 1, \ldots, n-1$, we have,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Let's write the **first few** equations explicitly,

$$\underbrace{-T_0 + 2T_1 - T_2}_{i=1} = \frac{h^2}{k} q(x_1)$$

$$\underbrace{-T_1 + 2T_2 - T_3}_{i=2} = \frac{h^2}{k} q(x_2)$$

$$\underbrace{-T_2 + 2T_3 - T_4}_{i=3} = \frac{h^2}{k} q(x_3)$$

From finite differences to a linear system (start with equations)

For each interior grid point $i = 1, \ldots, n-1$, we have,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Let's write the **first few** equations explicitly,

$$\underbrace{-T_0 + 2T_1 - T_2}_{i=1} = \frac{h^2}{k} q(x_1)$$

$$\underbrace{-T_1 + 2T_2 - T_3}_{i=2} = \frac{h^2}{k} q(x_2)$$

$$\underbrace{-T_2 + 2T_3 - T_4}_{i=3} = \frac{h^2}{k} q(x_3)$$

Pattern: each equation couples a point with its two neighbors.

Separate known vs unknown: interior temperatures

Boundary temperatures are known,

$$T(x_0 = 0) = T_0 = T_a, \qquad T(x_n = L) = T_n = T_b.$$

Separate known vs unknown: interior temperatures

Boundary temperatures are known,

$$T(x_0 = 0) = T_0 = T_a, \qquad T(x_n = L) = T_n = T_b.$$

Unknowns are the **interior** values,

$$T_1,\ T_2,\ \ldots,\ T_{n-1}.$$

Separate known vs unknown: interior temperatures

Boundary temperatures are known,

$$T(x_0 = 0) = T_0 = T_a, \qquad T(x_n = L) = T_n = T_b.$$

Unknowns are the **interior** values,

$$T_1,\ T_2,\ \ldots,\ T_{n-1}.$$

Stack the interior unknowns into a vector,

$$\mathbf{T} = \begin{bmatrix} T_1 \\ T_2 \\ \vdots \\ T_{n-1} \end{bmatrix}.$$

We will solve for all interior temperatures at once using matrix form.

Boundary conditions move to the right-hand side

Only the **first** and **last** interior equations touch the boundaries.

Boundary conditions move to the right-hand side

Only the **first** and **last** interior equations touch the boundaries.

For $i = 1$:

$$-T_0 + 2T_1 - T_2 = \frac{h^2}{k} q(x_1)$$

Boundary conditions move to the right-hand side

Only the **first** and **last** interior equations touch the boundaries.

For $i = 1$:

$$-T_0 + 2T_1 - T_2 = \frac{h^2}{k} q(x_1)$$

Move the known boundary value T_0 to the right:

$$2T_1 - T_2 = \frac{h^2}{k} q(x_1) + T_0.$$

Boundary conditions move to the right-hand side

Only the **first** and **last** interior equations touch the boundaries.

For $i = 1$:

$$-T_0 + 2T_1 - T_2 = \frac{h^2}{k} q(x_1)$$

Move the known boundary value T_0 to the right:

$$2T_1 - T_2 = \frac{h^2}{k} q(x_1) + T_0.$$

For $i = n - 1$:

$$-T_{n-2} + 2T_{n-1} - T_n = \frac{h^2}{k} q(x_{n-1})$$

Boundary conditions move to the right-hand side

Only the **first** and **last** interior equations touch the boundaries.

For $i = 1$:

$$-T_0 + 2T_1 - T_2 = \frac{h^2}{k} q(x_1)$$

Move the known boundary value T_0 to the right:

$$2T_1 - T_2 = \frac{h^2}{k} q(x_1) + T_0.$$

For $i = n - 1$:

$$-T_{n-2} + 2T_{n-1} - T_n = \frac{h^2}{k} q(x_{n-1})$$

Move the known boundary value T_n to the right:

$$-T_{n-2} + 2T_{n-1} = \frac{h^2}{k} q(x_{n-1}) + T_n.$$

Boundary conditions move to the right-hand side

Only the **first** and **last** interior equations touch the boundaries.

For $i = 1$:

$$-T_0 + 2T_1 - T_2 = \frac{h^2}{k} q(x_1)$$

Move the known boundary value T_0 to the right:

$$2T_1 - T_2 = \frac{h^2}{k} q(x_1) + T_0.$$

For $i = n - 1$:

$$-T_{n-2} + 2T_{n-1} - T_n = \frac{h^2}{k} q(x_{n-1})$$

Move the known boundary value T_n to the right:

$$-T_{n-2} + 2T_{n-1} = \frac{h^2}{k} q(x_{n-1}) + T_n.$$

Message: boundary values become part of the forcing term.

Stack all equations: $A\mathbf{T} = \mathbf{b}$

Writing all interior equations together gives a linear system,

$$A\mathbf{T} = \mathbf{b}.$$

Stack all equations: $A\mathbf{T} = \mathbf{b}$

Writing all interior equations together gives a linear system,

$$A\mathbf{T} = \mathbf{b}.$$

Matrix A

$$\begin{bmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -1 & 2 & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{bmatrix}$$

$\mathbf{T}$

$$\begin{bmatrix} T_1 \\ T_2 \\ \vdots \\ T_{n-1} \end{bmatrix}$$

$\mathbf{b}$

$$\begin{bmatrix} \frac{h^2}{k} q(x_1) + T_0 \\ \frac{h^2}{k} q(x_2) \\ \vdots \\ \frac{h^2}{k} q(x_{n-2}) \\ \frac{h^2}{k} q(x_{n-1}) + T_n \end{bmatrix}$$

Stack all equations: $A\mathbf{T} = \mathbf{b}$

Writing all interior equations together gives a linear system,

$$A\mathbf{T} = \mathbf{b}.$$

Matrix A

$$\begin{bmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -1 & 2 & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{bmatrix}$$

$\mathbf{T}$

$$\begin{bmatrix} T_1 \\ T_2 \\ \vdots \\ T_{n-1} \end{bmatrix}$$

$\mathbf{b}$

$$\begin{bmatrix} \frac{h^2}{k} q(x_1) + T_0 \\ \frac{h^2}{k} q(x_2) \\ \vdots \\ \frac{h^2}{k} q(x_{n-2}) \\ \frac{h^2}{k} q(x_{n-1}) + T_n \end{bmatrix}$$

Each row of A corresponds to one interior grid point equation.

Solving $A\mathbf{T} = \mathbf{b}$ using PyTorch

We have reduced our boundary value problem to a linear system,

$$A\mathbf{T} = \mathbf{b}.$$

Solving $A\mathbf{T} = \mathbf{b}$ using PyTorch

We have reduced our boundary value problem to a linear system,

$$A\mathbf{T} = \mathbf{b}.$$

In PyTorch, solving a linear system is one line,

```
import torch
# A : matrix from finite differences
# b : right-hand side vector
T = torch.linalg.solve(A, b)
```

Solving $A\mathbf{T} = \mathbf{b}$ using PyTorch

We have reduced our boundary value problem to a linear system,

$$A\mathbf{T} = \mathbf{b}.$$

In PyTorch, solving a linear system is one line,

```
import torch
# A : matrix from finite differences
# b : right-hand side vector
T = torch.linalg.solve(A, b)
```

Interpretation:

- A encodes how grid points interact
- $\mathbf{b}$ contains sources + boundary conditions
- $\mathbf{T}$ gives interior temperatures

Solving $A\mathbf{T} = \mathbf{b}$ using PyTorch

We have reduced our boundary value problem to a linear system,

$$A\mathbf{T} = \mathbf{b}.$$

In PyTorch, solving a linear system is one line,

```
import torch
# A : matrix from finite differences
# b : right-hand side vector
T = torch.linalg.solve(A, b)
```

Interpretation:

- A encodes how grid points interact
- $\mathbf{b}$ contains sources + boundary conditions
- $\mathbf{T}$ gives interior temperatures

Physics → discretization → linear algebra → solution

Example 2: Steady heat in 1D (with Neumann BC)

Same ODE as Example 1,

$$-k\,T''(x) = q(x), \qquad x \in [0, L].$$

Example 2: Steady heat in 1D (with Neumann BC)

Same ODE as Example 1,

$$-k\,T''(x) = q(x), \qquad x \in [0, L].$$

But now the boundary conditions prescribe the slope (flux):

$$T'(0) = \alpha, \qquad T'(L) = \beta.$$

Example 2: Steady heat in 1D (with Neumann BC)

Same ODE as Example 1,

$$-k\,T''(x) = q(x), \qquad x \in [0, L].$$

But now the boundary conditions prescribe the slope (flux):

$$T'(0) = \alpha, \qquad T'(L) = \beta.$$

Physical interpretation

Neumann BCs specify **heat flux** at the boundaries (insulated / imposed flux), instead of fixing the temperature values.

Discretize the rod (same grid as before)

Take the same grid points:

$$x_i = ih, \qquad i = 0, 1, \ldots, n, \qquad h = \frac{L}{n}.$$

Discretize the rod (same grid as before)

Take the same grid points:

$$x_i = ih, \qquad i = 0, 1, \ldots, n, \qquad h = \frac{L}{n}.$$

Unknowns are still:

$$T_i \approx T(x_i).$$

Discretize the rod (same grid as before)

Take the same grid points:

$$x_i = ih, \qquad i = 0, 1, \ldots, n, \qquad h = \frac{L}{n}.$$

Unknowns are still:

$$T_i \approx T(x_i).$$

Key difference from Dirichlet:

- Previously T_0 and T_n were known numbers.

Discretize the rod (same grid as before)

Take the same grid points:

$$x_i = ih, \qquad i = 0, 1, \ldots, n, \qquad h = \frac{L}{n}.$$

Unknowns are still:

$$T_i \approx T(x_i).$$

Key difference from Dirichlet:

- Previously T_0 and T_n were known numbers.
- Now we do *not* know T_0 or T_n; instead we know the slopes.

Discretize the rod (same grid as before)

Take the same grid points:

$$x_i = ih, \qquad i = 0, 1, \ldots, n, \qquad h = \frac{L}{n}.$$

Unknowns are still:

$$T_i \approx T(x_i).$$

Key difference from Dirichlet:

- Previously T_0 and T_n were known numbers.
- Now we do *not* know T_0 or T_n; instead we know the slopes.

So we keep **all** nodal values as unknowns:

$$T_0,\ T_1,\ \ldots,\ T_n.$$

Interior finite difference equations (unchanged)

For interior points $i = 1, \ldots, n-1$ (same as Example 1):

$$T''(x_i) \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}.$$

Interior finite difference equations (unchanged)

For interior points $i = 1, \ldots, n-1$ (same as Example 1):

$$T''(x_i) \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}.$$

Plug into $-kT''(x) = q(x)$:

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Interior finite difference equations (unchanged)

For interior points $i = 1, \ldots, n-1$ (same as Example 1):

$$T''(x_i) \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{h^2}.$$

Plug into $-kT''(x) = q(x)$:

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Message: only the *boundary treatment* changes.

Finite differences for Neumann boundary conditions

Neumann BCs prescribe derivatives at boundaries.

Finite differences for Neumann boundary conditions

Neumann BCs prescribe derivatives at boundaries.

Left boundary ($x = 0$):

$$T'(0) = \alpha \quad \approx \quad \frac{T_1 - T_0}{h} = \alpha \quad \Rightarrow \quad -T_0 + T_1 = h\alpha.$$

Finite differences for Neumann boundary conditions

Neumann BCs prescribe derivatives at boundaries.

Left boundary ($x = 0$):

$$T'(0) = \alpha \quad \approx \quad \frac{T_1 - T_0}{h} = \alpha \quad \Rightarrow \quad -T_0 + T_1 = h\alpha.$$

Right boundary ($x = L$):

$$T'(L) = \beta \quad \approx \quad \frac{T_n - T_{n-1}}{h} = \beta \quad \Rightarrow \quad -T_{n-1} + T_n = h\beta.$$

Finite differences for Neumann boundary conditions

Neumann BCs prescribe derivatives at boundaries.

Left boundary ($x = 0$):

$$T'(0) = \alpha \quad \approx \quad \frac{T_1 - T_0}{h} = \alpha \quad \Rightarrow \quad -T_0 + T_1 = h\alpha.$$

Right boundary ($x = L$):

$$T'(L) = \beta \quad \approx \quad \frac{T_n - T_{n-1}}{h} = \beta \quad \Rightarrow \quad -T_{n-1} + T_n = h\beta.$$

Boundary conditions become equations

Instead of plugging known values into **b**, we add two new linear equations.

Linear system structure (what changes vs Dirichlet?)

Unknown vector now includes all grid values,

$$\mathbf{T} = \begin{bmatrix} T_0 \\ T_1 \\ \vdots \\ T_{n-1} \\ T_n \end{bmatrix}.$$

Linear system structure (what changes vs Dirichlet?)

Unknown vector now includes all grid values,

$$\mathbf{T} = \begin{bmatrix} T_0 \\ T_1 \\ \vdots \\ T_{n-1} \\ T_n \end{bmatrix}.$$

Interior rows (for $i = 1, \ldots, n-1$) keep the same tridiagonal stencil,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

Linear system structure (what changes vs Dirichlet?)

Unknown vector now includes all grid values,

$$\mathbf{T} = \begin{bmatrix} T_0 \\ T_1 \\ \vdots \\ T_{n-1} \\ T_n \end{bmatrix}.$$

Interior rows (for $i = 1, \ldots, n-1$) keep the same tridiagonal stencil,

$$-T_{i-1} + 2T_i - T_{i+1} = \frac{h^2}{k}\, q(x_i).$$

But the **first and last rows** come from Neumann BCs:

$$-T_0 + T_1 = h\alpha, \qquad -T_{n-1} + T_n = h\beta.$$

Same building blocks as Example 1, just different boundary rows.

Linear system structure (what changes vs Dirichlet?)

$$A\mathbf{T} = \mathbf{b}.$$

All nodal temperatures are now unknown,

Matrix A

$$\begin{bmatrix} -1 & 1 & 0 & \cdots & 0 & 0 \\ -1 & 2 & -1 & \ddots & & \vdots \\ 0 & -1 & 2 & -1 & & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & & -1 & 2 & -1 & 0 \\ 0 & 0 & \cdots & 0 & -1 & 1 \end{bmatrix}$$

T

$$\begin{bmatrix} T_0 \\ T_1 \\ \vdots \\ T_{n-1} \\ T_n \end{bmatrix}$$

b

$$\begin{bmatrix} h\alpha \\ \frac{h^2}{k} q(x_1) \\ \vdots \\ \frac{h^2}{k} q(x_{n-1}) \\ h\beta \end{bmatrix}$$

Linear system structure (what changes vs Dirichlet?)

$$A\mathbf{T} = \mathbf{b}.$$

All nodal temperatures are now unknown,

Matrix A

$$\begin{bmatrix} -1 & 1 & 0 & \cdots & 0 & 0 \\ -1 & 2 & -1 & \ddots & & \vdots \\ 0 & -1 & 2 & -1 & & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & & -1 & 2 & -1 & 0 \\ 0 & 0 & \cdots & 0 & -1 & 1 \end{bmatrix}$$

T

$$\begin{bmatrix} T_0 \\ T_1 \\ \vdots \\ T_{n-1} \\ T_n \end{bmatrix}$$

b

$$\begin{bmatrix} h\alpha \\ \frac{h^2}{k} q(x_1) \\ \vdots \\ \frac{h^2}{k} q(x_{n-1}) \\ h\beta \end{bmatrix}$$

First and last rows come from Neumann boundary conditions.

Important note

Pure Neumann problems

If both boundaries are Neumann, the solution is determined only up to an additive constant (adding a constant does not change derivatives).

Important note

Pure Neumann problems

If both boundaries are Neumann, the solution is determined only up to an additive constant (adding a constant does not change derivatives).

In practice, we fix a reference (e.g., specify $T(0)$ or the mean temperature) to make the solution unique.

Mini takeaways

- BVP $\Rightarrow$ we discretized space (not time)
- Finite differences convert derivatives into linear algebra
- BVP discretization typically gives a **system** of equations
- This is why linear algebra matters in this course

Mini takeaways

- BVP $\Rightarrow$ we discretized space (not time)
- Finite differences convert derivatives into linear algebra
- BVP discretization typically gives a **system** of equations
- This is why linear algebra matters in this course

We have turned calculus into a matrix equation.

Common misconception: BVPs are **not** only for second-order ODEs

Example of a first-order BVP,

$$y'(x) = f(x), \qquad y(0) = a, \quad y(L) = b.$$

Common misconception: BVPs are **not** only for second-order ODEs

Example of a first-order BVP,

$$y'(x) = f(x), \qquad y(0) = a, \quad y(L) = b.$$

This may or may not have a solution depending on compatibility,

$$b - a \stackrel{?}{=} \int_0^L f(x)\,\mathrm{d}x.$$

Common misconception: BVPs are **not** only for second-order ODEs

Example of a first-order BVP,

$$y'(x) = f(x), \qquad y(0) = a, \quad y(L) = b.$$

This may or may not have a solution depending on compatibility,

$$b - a \stackrel{?}{=} \int_0^L f(x)\,\mathrm{d}x.$$

Why second order is common in physics: many equilibrium laws involve curvature (second derivative) and fluxes.

Summary: what to remember

Big idea: Boundary Value Problems (BVPs)

Today we learned how spatial differential equations are solved computationally.

Summary: what to remember

Big idea: Boundary Value Problems (BVPs)

Today we learned how spatial differential equations are solved computationally.

Key steps:

1. Physical model $\rightarrow$ differential equation

Summary: what to remember

Big idea: Boundary Value Problems (BVPs)

Today we learned how spatial differential equations are solved computationally.

Key steps:

1. Physical model $\rightarrow$ differential equation
2. Discretize the domain using finite differences

Summary: what to remember

Big idea: Boundary Value Problems (BVPs)

Today we learned how spatial differential equations are solved computationally.

Key steps:

1. Physical model $\rightarrow$ differential equation
2. Discretize the domain using finite differences
3. Replace derivatives with algebraic approximations

Summary: what to remember

Big idea: Boundary Value Problems (BVPs)

Today we learned how spatial differential equations are solved computationally.

Key steps:

1. Physical model $\rightarrow$ differential equation
2. Discretize the domain using finite differences
3. Replace derivatives with algebraic approximations
4. Obtain a linear system $A\mathbf{T} = \mathbf{b}$

Summary: what to remember

Big idea: Boundary Value Problems (BVPs)

Today we learned how spatial differential equations are solved computationally.

Key steps:

1. Physical model $\rightarrow$ differential equation
2. Discretize the domain using finite differences
3. Replace derivatives with algebraic approximations
4. Obtain a linear system $A\mathbf{T} = \mathbf{b}$
5. Solve using numerical linear algebra tools

$$\textit{Calculus problem} \Rightarrow \textit{Linear algebra problem}$$

Next lecture: From BVPs to Implicit Methods

So far:

- IVPs were solved by stepping forward in time
- BVPs required solving a globally coupled system of equations

Next question:

What happens when time-stepping methods also become globally coupled?

Next lecture(s):

- Motivation for implicit methods
- Stability limitations of explicit methods
- Stiff differential equations
- Backward Euler method
- Implicit Midpoint method
- Solving nonlinear equations arising from implicit schemes

Next lecture: From BVPs to Implicit Methods

So far:

- IVPs were solved by stepping forward in time
- BVPs required solving a globally coupled system of equations

Next question:

What happens when time-stepping methods also become globally coupled?

Next lecture(s):

- Motivation for implicit methods
- Stability limitations of explicit methods
- Stiff differential equations
- Backward Euler method
- Implicit Midpoint method
- Solving nonlinear equations arising from implicit schemes

Implicit methods connect time-stepping with ideas we saw in BVPs.